

MLOps 与模型生命周期

从一个 Notebook 里的实验模型，到稳定服务线上业务的智能系统，中间横亘着工程化、自动化、可观测性三大鸿沟。MLOps (Machine Learning Operations) 正是弥合这道鸿沟的方法论与工具体系——它把 DevOps 系统中，让模型像软件一样可重复构建、可追踪、可回滚、可监控。

MLOps 与模型生命周期

从一个 Notebook 里的实验模型，到稳定服务线上业务的智能系统，中间横亘着工程化、自动化、可观测性三大鸿沟。MLOps (Machine Learning Operations) 正是弥合这道鸿沟的方法论与工具体系——它把 DevOps 系统中，让模型像软件一样可重复构建、可追踪、可回滚、可监控。

- - -

一、MLOps 的定义与成熟度模型

1.1 什么是 MLOps

MLOps 是一种工程文化和实践集合，目标是在 ML 系统的全生命周期内实现自动化、可重复、可审计的流程。它不仅包含代码，还包含数据、模型、训练配置、评估脚本等。

与 DevOps 的关键区别：

维度 DevOps MLOps

制品类型	代码 + 二进制	代码 + 数据 + 模型 + 配置
版本对象	代码版本	数据版本 + 模型版本 + 代码版本
质量门禁	单测 + 集成测试	指标达标 + 数据校验 + 偏见检查
衰退原因	代码 Bug	数据漂移 + 概念漂移
部署单元	服务 / 容器	模型权重 + 推理服务
回滚方式	代码回滚	模型权重回滚 + 数据回滚

1.2 Google MLOps 成熟度模型

Google 提出的三级成熟度模型是业界广泛引用的参考框架：

级别 名称 特征 自动化范围

Level 0 手动流程 Notebook 手动训练、手动部署、无监控 几乎无自动化

Level 1 ML 流水线自动化 训练流水线可重复触发、模型注册中心、特征源可控 训练自动化

Level 2 CI / CD 流水线自动化 代码 / 数据变更自动触发训练、自动评估、自动部署、持续监控

Level 0 在原型阶段可用，但任何严肃的生产系统都应至少达到 Level 1。Level 2 适合模型迭代频繁、对线上质量要求高的业务（如推荐、风控）。

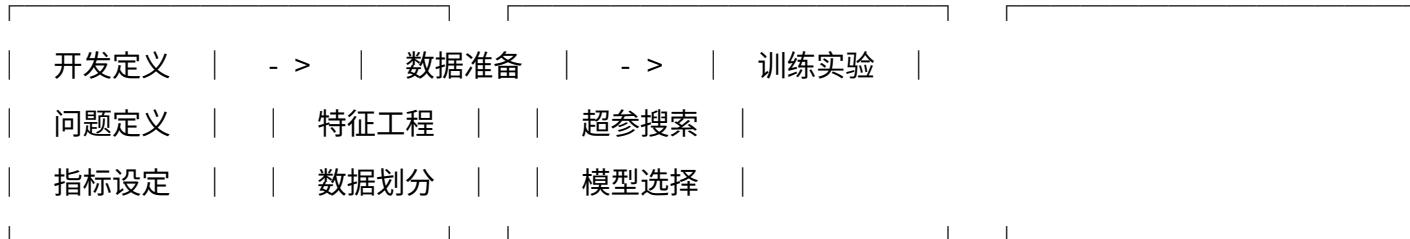
- - -

二、模型生命周期（开发 - 训练 - 评估 - 部署 - 监控 - 迭代）

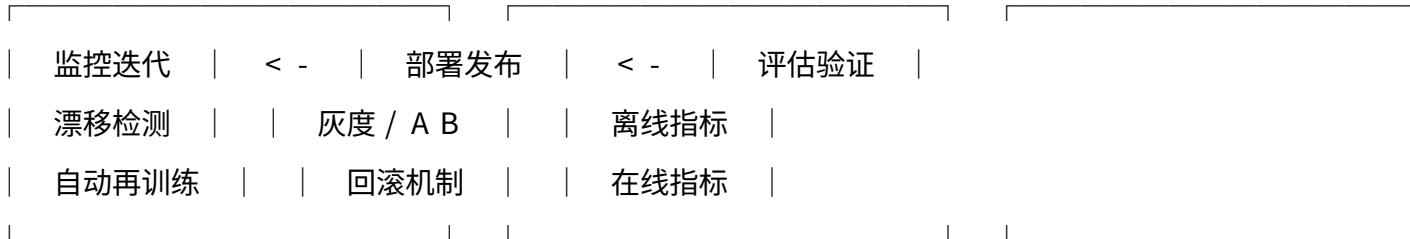
模型生命周期是一个闭环而非线性流程，每次迭代都从监控信号或业务需求出发，最终以新版本部署为终点。

业务需求 / 数据更新 / 监报告警

|



|



各阶段的关键产出：

阶段 关键产出 关键风险

开发 问题定义文档、指标基线 目标错配（优化指标 ≠ 业务目标）

训练 模型权重、训练日志 数据泄漏、过拟合

评估 评估报告、对比基线 评估集分布与线上不一致

部署 推理服务、版本号 部署环境与训练环境不一致

监控 漂移报告、性能曲线 监控指标滞后于业务损失

迭代 新版本、变更说明 反复打补丁而非根因修复

- - -

三、实验管理（MLflow、Weights & Biases、TensorBoard）

实验管理的核心需求是：可追踪、可对比、可复现。当团队同时进行数十个超参组合的实验时，没有实验追踪系统几乎无法工作。

3.1 主流工具对比

工具 定位 优势 劣势

MLflow 开源全流程平台 语言无关、自托管、模型注册一体化 UI 较朴素、协作功能弱

Weights & Biases (W&B) 商业 SaaS 实验追踪 UI 强大、团队协作、可视

TensorBoard 单机可视化 与 TF/PyTorch 深度集成、零成本 不适合团队协作、无

Comet ML 商业实验追踪 实验对比功能强 生态不及 W&B

Aim 开源实验追踪 处理海量实验性能好 社区较小

3.2 MLflow 实验追踪示例

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

mlflow.set_experiment("churn-prediction-v2")

with mlflow.start_run(runname="rfn500depth10"):
    # 自动记录 sklearn 模型
    mlflow.autolog()

    # 手动记录超参
    params = {"nestimators": 500, "maxdepth": 10, "ra
    mlflow.logparams(params)

    # 训练
```

```

model = RandomForestClassifier(params)
model.fit(Xtrain, ytrain)

# 评估并记录指标
ypred = model.predict(Xval)
mlflow.logmetrics({
    "accuracy": accuracyscore(yval, ypred),
    "f1": f1score(yval, ypred, average="macro")
})

# 注册模型到 Model Registry
mlflow.register_model(
    f"runs:/{mlflow.active_run().info.runid}/model",
    "ChurnPredictor"
)

```

实验管理应遵循的纪律：任何线上模型必须可追溯到训练它的代码、数据、超参和实验运行 ID。

- - -

四、模型注册与版本管理

模型注册中心 (Model Registry) 是 MLOps 的 "代码仓库", 集中管理模型元数据、版本

4.1 典型状态机

```

None -> Staging -> Production -> Archived
↑   |
└───┘

```

回滚

状态 含义 触发条件

None 刚注册, 未评估 实验完成并注册
 Staging 预发布环境验证 离线指标达标
 Production 线上服务 灰度验证通过
 Archived 下线归档 被新版本替代或失效

4.2 模型元数据应包含

- 模型版本号（语义化：major.minor.patch）
- 训练数据版本（DVC / 数据快照）
- 代码 commit hash
- 超参与训练配置
- 评估指标与评估集 hash
- 训练者、时间、环境
- 部署状态与目标环境

DVC 数据版本管理示例

```
git add data/train.csv
dvc add data/train.csv
dvc push # 推送到对象存储
git commit -m "feat: 更新训练集 v3"
```

拉取指定版本数据

```
git checkout v2.1
dvc pull
```

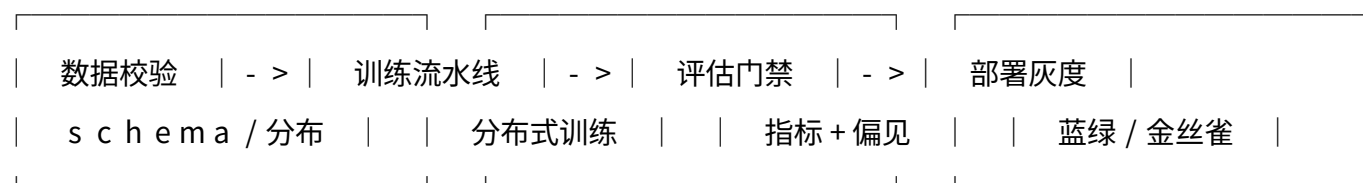
五、CI/CD for ML（持续训练、持续部署）

ML 的 CI/CD 不仅是代码 CI/CD，还包含持续训练（CT，Continuous Training）——数据或代码变更自动触发训练流水线。

5.1 ML CI/CD 流水线典型阶段

代码 / 数据 / 调度触发

|



| 不达标



流水线中断 + 告警

5.2 GitHub Actions 自动训练示例

```
name: retrain-on-data-update
on:
  push:
  paths:
    - 'data/train.csv'
jobs:
  train:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
      with: { python-version: '3.11' }
      - name: Install deps
        run: pip install -r requirements.txt
      - name: Validate data schema
        run: python scripts/validatedata.py --input data/
      - name: Train model
        run:
          python train.py --output models/
        echo "MODELVERSION=$(git rev-parse --short HEAD)"
      - name: Evaluate & gate
        run: python scripts/evalgate.py --threshold 0.92
      - name: Register & deploy
        if: success()
        run:
          python scripts/registermodel.py --version $MODELVERSION
          python scripts/deploycanary.py --percent 5
```

5.3 评估门禁设计

门禁类型 判定逻辑 失败动作

性能门禁 新模型指标 $\geq$ 基线 $\times (1 - tolerance)$ 阻断部署

偏见门禁 各群体指标差异 $<$ 阈值 阻断 + 人工复审

延迟门禁 p99 推理延迟 $<$ SLA 阻断部署

回归门禁 旧评估集指标不下降 阻断 + 告警

- - -

六、模型服务架构（在线推理、批量推理、边缘部署）

6.1 三种典型部署模式

模式 延迟要求 吞吐特征 典型实现

在线推理 毫秒级 低并发持续 Triton、TorchServe、vLLM

批量推理 分钟 - 小时级 高吞吐 Spark + 模型 UDF、Batch Job

流式推理 秒级 持续流 Flink + 推理算子

边缘部署 本地毫秒级 单设备 TFLite、ONNX Runtime Mobile

6.2 在线推理服务架构

客户端 -> API 网关 -> 负载均衡 -> 推理 Pod（多副本）

|

├─ 模型缓存（本地权重）

├─ 请求批处理（dynamic batching）

└─ 指标采集（Prometheus）

关键优化技术：

- 动态批处理（Dynamic Batching）：将短时间内的多个请求合并为一个 batch，提升推理效率
- 模型预热（Warm-up）：服务启动时先跑 dummy 输入，避免首个请求超时
- 多模型混部：同一 GPU 上多个模型共享显存，提高利用率
- 分级缓存：相同输入的结果缓存（embedding 服务常用）

```

Triton 动态批处理配置 config.pbtxt
dynamicbatching {
  preferredbatchsize: [4, 8, 16]
  maxqueuedelaymicroseconds: 100000 # 100ms 内尽量凑批
  preserveordering: true
}

- - -

```

七、模型监控（数据漂移、概念漂移、性能衰减）

模型一旦上线就开始 " 老化 " 。监控是发现老化并触发迭代的神经系统。

7.1 三类关键漂移

类型 定义 检测方法

数据漂移 (Data Drift) 输入分布 $P(X)$ 变化 KS 检验、PSI、Wasserstein

概念漂移 (Concept Drift) $P(Y|X)$ 变化 残差监控、ADWIN、DDM

标签漂移 * $P(Y)$ 变化 标签分布统计

7.2 常用监控指标

```

import numpy as np
from scipy.stats import ks2samp

def psi(expected, actual, bins=10):
    """Population Stability Index"""
    expectedpct = np.histogram(expected, bins=bins)[0]
    actualpct = np.histogram(actual, bins=bins)[0] /
    # 避免 0
    expectedpct = np.clip(expectedpct, 1e-4, None)
    actualpct = np.clip(actualpct, 1e-4, None)
    return np.sum((actualpct - expectedpct) * np.log(actualpct / expectedpct))

```

PSI < 0.1 稳定；0.1 - 0.25 关注；> 0.25 显著漂移

7.3 监控指标分层

层级 指标 触发动作

基础设施 CPU / GPU 利用率、显存、QPS 自动扩缩容

服务质量 延迟、错误率、超时率 告警 + 重试

模型性能 在线指标（有延迟标签） / 代理指标 触发再训练

数据质量 schema、缺失率、分布漂移 阻断上线 + 排查

注意：很多业务标签有延迟（如信贷违约 30 天后才知道），需要使用代理指标（proxy metric）即时估计性能。

- - -

八、A / B 测试与灰度发布

8.1 灰度发布阶段

Shadow（影子） -> Canary（金丝雀 1 - 5%） -> Ramp（10 - 50%） -> Full

- Shadow：新模型接收流量但不返回结果，仅对比输出
- Canary：少量真实流量，重点观察错误率与延迟
- Ramp：逐步扩大流量，统计显著性验证
- Full：全量切换，保留快速回滚能力

8.2 A / B 测试统计设计

```
import numpy as np
from statsmodels.stats.proportion import proportion_ztest
```

A 组（旧模型）转化 980 / 10000

B 组（新模型）转化 1050 / 10000

```
counts = np.array([980, 1050])
```

```
nobs = np.array([10000, 10000])
```

```
zstat, pval = proportion_ztest(counts, nobs)
```

```
print(f"p-value = {pval:.4f}")
```

p < 0.05 视为显著差异

A / B 测试常见陷阱：

- 1 . 样本量不足：未做 M D E 计算，提早下结论
- 2 . p e e k i n g p r o b l e m : 频繁查看导致假阳性
- 3 . 短期效应：新模型短期新鲜感强，长期衰减
- 4 . 交互效应：多实验并行互相污染

- - -

九、MLOps 工具链对比

环节 主流工具 选型建议

数据版本 D V C、L a k e F S、D e l t a L a k e 中小项目 D V C ; 数据湖 D e l t a

特征存储 F e a s t、T e c t o n、H o p s w o r k s 离线+在线一致性需求强时使用

实验追踪 M L f l o w、W & B、A i m 个人/小团队 M L f l o w ; 强协作 W & B

模型注册 M L f l o w R e g i s t r y、V e r t e x A I、M o d e l D B 与实验追踪同源更省事

编排 K u b e f l o w、A i r f l o w、A r g o、P r e f e c t 数据团队 A i r f l o w ; M L

C I / C D G i t H u b A c t i o n s、G i t L a b C I、J e n k i n s 通用 C I 工具即可

服务化 T r i t o n、T o r c h S e r v e、v L L M、B e n t o M L L L M 选 v L L M ; 通用选

监控 E v i d e n t l y、A r i z e、W h y L a b s、F i d d l e r 漂移监控选 E v i d e n t l y

端到端平台 K u b e f l o w、V e r t e x A I、S a g e M a k e r、A z u r e M L 自建选 K

选型原则

- 1 . 从 L e v e l 1 起步：先把实验追踪 + 模型注册做起来，再谈 C I / C D
- 2 . 避免过早平台化：先用开源组件拼装，验证流程后再引入端到端平台
- 3 . 工具链解耦：实验追踪、训练编排、服务、监控尽量可独立替换
- 4 . 拥抱标准：优先支持 O N N X、M L f l o w M o d e l F o r m a t 等开放格式

- - -

十、关键词

MLOps , 模型生命周期 , M L f l o w , C I / C D , 持续训练 , 模型注册中心 , 数据漂移 ,
A / B 测试 , 灰度发布 , 动态批处理 , 特征存储 , D V C